System Design Document: Cambridge Care Services AI Chatbot

Target Website: https://www.cambridgecareservices.co.uk/ | **Author:** Freelance AI Engineering Plan | **Status:** Ready for Implementation

---

1. Executive Summary & Objective

Cambridge Care Services is a regulated home care provider registered with the Care Quality Commission (CQC) operating across Cambridge and South Cambridgeshire. The objective of this project is to create an intelligent, domain-specific AI Chatbot using **Retrieval-Augmented Generation (RAG)** to:

- **24/7 Client Guidance:** Instantly answer inquiries on care services, postcodes, fees, and funding options.
- **Lead Generation Engine:** Convert website visitors into direct care inquiries by capturing names and phone numbers for free home assessments.
- **Recruitment Pre-Screening:** Provide immediate guidance for carer applicants on DBS, driving requirements, and application procedures.
- **Seamless Website Integration:** Embed via an external script tag without needing access to the client's internal codebase or servers.

2. Overall Architecture Blueprint

The project follows a decoupled, two-stage architecture: local standalone testing with **Streamlit**, followed by cloud API deployment with **FastAPI** and external website widget integration.

Component	Technology	Role & Location
Frontend UI (Testing)	Streamlit (app.py)	Local visual chat interface on developer laptop. Rapid iteration with zero cloud complexity.
Knowledge Base	5 Clean .txt Docs	Services, Geographic Coverage (CB1-CB25), Onboarding/FAQs, Recruitment, and Leadership/Compliance.
Vector Database	ChromaDB / FAISS	Indexes document chunks into semantic embeddings. Runs locally in `./chroma_db` folder.
RAG Engine	Python (rag.py) + LLM	Retrieves relevant paragraphs and prompts LLM (OpenAI / Gemini) with strict guardrails.
Production API	FastAPI	Cloud backend service accepting JSON messages from the live website widget.
Live Website Widget	Lightweight JS / HTML	Single <script> tag pasted into the client's site. Displays floating chat bubble.

3. Component Breakdown & How It Works

A. The Knowledge Base (The 4 Source Documents)

- **01_about_and_services.txt:** CQC registration, office address (Deal Grove, Sawston), hours, 24/7 out-of-hours phone, and 9 care categories (Personal, Live-In, Dementia, Complex/Post-Hospital, Respite, Waking/Sleeping Night, Companionship, Supported Living, End-of-Life).

- **02_coverage_and_locations.txt:** Primary postcodes (CB1-CB5, CB21-CB25), covered villages (Sawston, Trumpington, Shelford, Linton, etc.), and out-of-area flexible assessment policy via phone.
- **03_process_and_faqs.txt:** 3-step onboarding (Consultation -> Free Home Assessment -> Care Plan), rapid 24-48h start, DBS checks, funding (Direct Payments, NHS Continuing Healthcare, Private), and sickness replacement guarantee.
- **04_careers_and_recruitment.txt:** Recruited roles, driving requirements, UK Right to Work, Care Certificate training, mileage allowance, and application submission channels.
- **05_leadership_funding_and_compliance.txt:** Registered Manager (Mr. Rudrayya Mathapati), CQC registration history (since March 2022), Cambridgeshire County Council assessments, Attendance Allowance, 999/111 emergency protocols, and review platforms.

B. The Vector Database (ChromaDB)

ChromaDB serves as the search index for the AI. It splits the documents into bite-sized semantic chunks and converts them into numerical vectors. When a user asks: *'Do you offer night care in Trumpington?'*, ChromaDB retrieves only the exact paragraphs concerning Night Support and Trumpington (CB2) in milliseconds, preventing hallucinations and keeping costs minimal.

C. Standalone Testing (Streamlit `app.py`)

For Phase 1 testing, everything runs locally on your laptop in **one unified Streamlit app**. Streamlit renders the chat screen in your browser and directly calls the RAG engine. **No cloud accounts, no API servers, and zero internet hosting are required during this stage.**

D. FastAPI & Cloud Hosting (Independent Architecture)

When moving to production, the Python RAG engine is wrapped in FastAPI and deployed onto a cloud server (e.g., Render, Railway, or AWS). **Key Fact:** The chatbot server does NOT need to be on the same cloud as the client's website. The live website communicates with your chatbot server over secure HTTPS, exactly like Google Maps or PayPal.

E. Live Website Integration (Zero Code Access Needed)

Because you do not have Cambridge Care Services' internal source code, integration is performed via a **standard JavaScript widget tag**. The client or webmaster simply pastes one line into their website header/footer:

```
<script src="https://your-bot-server.com/widget.js" defer></script>
```

4. Healthcare Guardrails & Lead Capture

- **Medical Advice Guardrail:** The bot will never prescribe, alter medication dosages, or give clinical diagnoses. It clarifies non-clinical home care boundaries and refers medical queries to GPs/District Nurses.
- **Geographic Precision:** For locations outside core South Cambridgeshire, it prompts the family to call 07917 611 425 for staffing availability rather than guessing.
- **Lead Capture Flow:** When a user asks about booking care, the bot offers: *'We would be delighted to arrange a free home assessment. Would you like to share your name and phone number so our team can call you?'* and records the lead.

5. Step-by-Step Implementation Roadmap

Phase	Action Items	Deliverable
Phase 1: Local Prototype (Today)	1. Create local virtual environment. 2. Ingest 4 .txt files into ChromaDB. 3. Build Streamlit test UI (`app.py`). 4. Stress-test questions locally.	Working interactive chatbot running on your laptop.
Phase 2: Client Demo	Showcase live demo to Cambridge Care Services to pitch freelance value and collect feedback.	Freelance contract approval & client sign-off.
Phase 3: Cloud API	1. Wrap RAG engine in FastAPI (`main.py`). 2. Deploy to 24/7 cloud server (e.g. Render/AWS). 3. Build `widget.js` embeddable script.	Live cloud endpoint with public widget script.
Phase 4: Live Launch	Client adds one <script> tag to their live website. Verify chat bubble on desktop and mobile.	Fully deployed, live production chatbot.